The wonderful & weird world of FLOATING points

Or why is 0.1 + 0.2 ≠ 0.3?

Peter Schmidt, March 2026

Why talk about floating points?

- How are floating points built?
- What are their limitations?
- Can they cause any problems?
- New(er) formats for floating points

How is this session structured?

I. Introduction

Some motivation on floating points as a subject

II. Practical session and demo

III. Q&A

I. Introduction

It’s a numbers’ game

Integer numbers: 2, -39, 1059 …

- Relatively straight forward Decimal <-> Binary
- Limited by number of bits available

What about real numbers: 0.15, sin(45), sqrt(2), π, etc.

- Also limited by number of bits, **BUT**
- They require a bit more work
- And there may be other problems down the line…

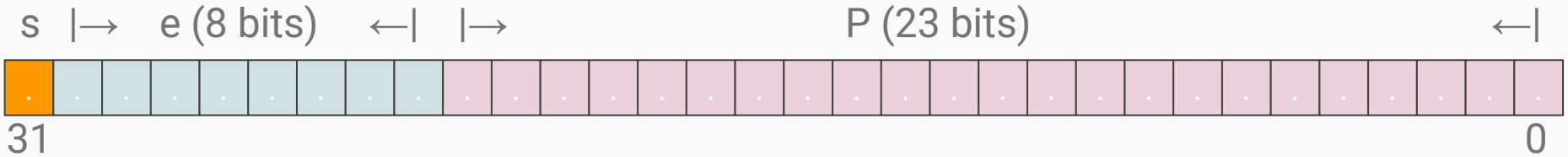
IEEE 754 Standard for floating point arithmetic

- **Published 1985**
- Latest revision: **2019**

3 parts to make a real number

1. **s** = sign ∓ - (0 ⇒positive, 1 ⇒negative)
2. **e** = Exponent (biased)
3. **p** = Mantissa (or significand)

32 bit floating point representation



What it looks like - simple example (32 bit)

$$(-1)^{sign} * 2^{exponent - bias\ (127)} * (1 + p)$$

sign

0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Number 2.0

sign

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Number 2 as integer

$(-1)^{sign} * 2^{exponent - bias\ (127)} * (1 + p)$

Example number 2.0 = $(+1) * 2^{1} * (1 + 0) = (-1)^{0} * 2^{128 - 127} * (1 + 0)$

Sign:

$(-1)^{0} = +1$

$(-1)^{1} = -1$

Mantissa (1+p):

The 1 is implicit (i.e. not encoded)

The value for $p = 0_{23}$, $\Sigma\ bit_n * 2^{-n}$

Exponent:

(b) 10000000 = (d) 128

From which the bias (d) 127 is subtracted.

=> exponent = 1

But that’s not all

- Some numbers have **no** finite representation
- Examples in **decimal**
  - ⅓ = 0.3333333… , ⅔ = 0.66666…

- Examples in **binary**
  - 0.1 = 0.**0011**0011001100110011…. (endless repeating sequence)

So what do we do?

We Round!

0	0	1	1	1	1	0	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	1

Repeating sequence

Error through rounding : 0.00000000149011611938476562 5

Binary value of 0.1 is (*slightly*) bigger

Rounding is, where trouble starts

Examples (see link below):

- Ariane 5 exploded shortly after launch on 4/6/1996
  - Reason: converting double to int
- Patriot rocket target missed 1991
  - Rounding error

BEWARE

- When **converting** numbers to different data types
- Of the **limit** of precision for each data type
- Creeping rounding errors
  - (**tiny** errors may become **big** when repeated many times)

https://www-users.cse.umn.edu/~arnold/disasters/

II. Demo & Practice session

1. Build your own 32 bit floating point number (10 min)

0.5

Value for sign:

Value for exponent:

Value for mantissa:

Resulting bits:

-5.0

Value for sign:

Value for exponent:

Value for mantissa:

Resulting bits:

$$(-1)^{sign} * 2^{exponent - bias\ (127)} * (1 + p)$$

Exercise break

Expected strings

$$(-1)^{sign} * 2^{exponent - bias\ (127)} * (1 + p)$$

sign

0	0	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Number 0.5, e = 126, p = 0

sign

1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Number -5.0, e = 129, p = 0.25, $(-1) * 2^{2} * (1 + 2^{-2})$

2. Look up tools for floats

Used in this class

- https://float.exposed/0x40a00000
- https://bartaz.github.io/ieee754-visualization/

Also useful

- https://www.h-schmidt.net/FloatConverter/IEEE754.html
- https://evanw.github.io/float-toy/

3. Simple arithmetics - Python

0.1 + 0.2 == 0.3

True or False?

Print the actual decimal value in Python

4. When rounding errors go ‘catastrophic’

a = 3.141592653589793

b = 6.022e23

Should expect: a = (b + a) - b

Print what we really get

5. Breaking the law

b = 6.022e23

Term1 = b - b + 1

Term2 = 1 + b - b

Term1 == Term2 ?

6. When errors add up

Small errors add up

Add 0.1 ten times in a loop

a.) Print the result.

b.) Is there another way to do this?

7. Some Conclusions

- Rounding errors are **inherent** to floating point representations
- **Beware** of converting data types
- **Beware** of loops
- **Use** libraries like math, numpy or scipy for floating point arithmetics
- **Numerical recipes** help
- **What precision** do I need?

8. Finally - is bigger better?

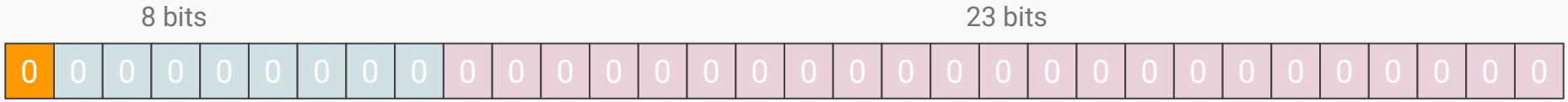
For High Performance Computing (HPC) there is a trend to go to

- Lower precision floating point data types
- It saves processing time, e.g. for AI/ML applications

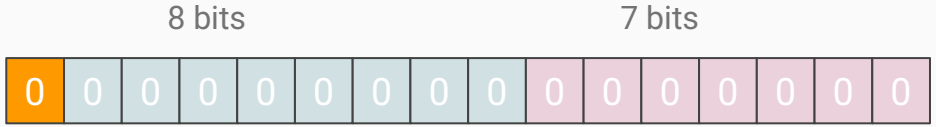
Examples: half precision (16 bit)

But which one: fp16 or bfloat16

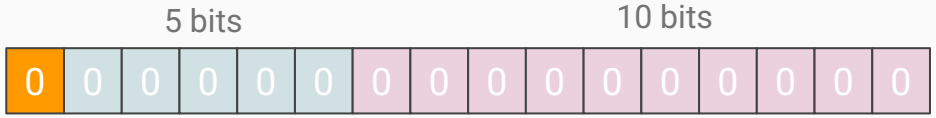
https://www.siam.org/publications/siam-news/articles/low-precision-floating-point-formats-the-wild-west-of-computer-arithmetic/



IEEE 754 single precision (32 bits)



bfloat16



IEEE 754 half precision

Floating points (in Python)

- https://docs.python.org/3/tutorial/floatingpoint.html
- https://docs.python.org/3/library/decimal.html
- https://realpython.com/python-fractions/#floating-point-and-decimal-numbers
- https://realpython.com/python-bitwise-operators/#binary-number-representations
- https://people.duke.edu/~ccc14/sta-663/ComputerArithmetic.html

More reading

- https://www.itu.dk/~sestoft/bachelor/IEEE754_article.pdf copy of the Daniel Goldberg paper
- https://arxiv.org/abs/2012.02492 revisiting the Goldberg paper, Vincent Lafage
- https://en.wikipedia.org/wiki/Floating-point_arithmetic for a “quick” read
- https://standards.ieee.org/ieee/754/6210/ the IEEE 754 standard (behind paywall)
- https://matloka.com/blog/floating-point-101

Lower Precision?
It’s a Wild West out there

- https://en.wikipedia.org/wiki/Bfloat16_floating-point_format an overview of some other floating point formats (incl lower precision)
- https://towardsdatascience.com/16-8-and-4-bit-floating-point-formats-how-does-it-work-d157a31ef2ef/ even lower bits floating points
- https://research.ibm.com/blog/low-precision-computing a paper by IBM why lower precision floating points may be enough
- https://developer.nvidia.com/blog/floating-point-8-an-introduction-to-efficient-lower-precision-ai-training/ NVidia on lower precision fps.
- https://indico.esa.int/event/445/contributions/8473/attachments/5559/9378/gernigon_cedric_EDHPC2023.pdf A presentation containing other low precision floating point formats

Thanks!

Contact me:

Peter Schmidt

peter@code4thought.org

https://pweschmidt.github.io/
https://github.com/pweschmidt
(*also on GitLab & Codeberg*)

LinkedIn
https://www.linkedin.com/in/pweschmidt/

BlueSky
@code4thought.bsky.social

Podcast Shows

https://github.com/pweschmidt/ByteSized-FloatingPoints